

Aarya Birwadkar

Batch C2 PRN 1262252502

Data Structures Assignment 8(circular queue)

Code:

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int queue[SIZE];
```

```
int front = -1, rear = -1;
```

```
void enqueue(int value)
```

```
{
```

```
    if ((rear + 1) % SIZE == front)
```

```
    {
```

```
        printf("Queue Overflow!\n");
```

```
        return;
```

```
    }
```

```
    if (front == -1)
```

```
        front = 0;
```

```
    rear = (rear + 1) % SIZE;
```

```
    queue[rear] = value;
```

```
}
```

```
void dequeue()
```

```
{
```

```
if (front == -1)
{
    printf("Queue Underflow!\n");
    return;
}
```

```
printf("Deleted: %d\n", queue[front]);
```

```
if (front == rear)
{
    front = -1;
    rear = -1;
}
else
{
    front = (front + 1) % SIZE;
}
}
```

```
void display()
{
    int i;

    if (front == -1)
    {
        printf("Queue is empty.\n");
        return;
    }
```

```
printf("Queue: ");
```

```
i = front;
```

```
while (1)
```

```
{
```

```
    printf("%d ", queue[i]);
```

```
    if (i == rear)
```

```
        break;
```

```
    i = (i + 1) % SIZE;
```

```
}
```

```
printf("\n");
```

```
}
```

```
int main()
```

```
{
```

```
    enqueue(10);
```

```
    enqueue(20);
```

```
    enqueue(30);
```

```
    enqueue(40);
```

```
    enqueue(50);
```

```
    display();
```

```
    dequeue();
```

```
    dequeue();
```

```

display();

enqueue(60);
enqueue(70);

display();

return 0;
}

```

Output:

dell@dell-Latitude-5420:~\$ gedit ass8.c

^C

dell@dell-Latitude-5420:~\$ gcc ass8.c

dell@dell-Latitude-5420:~\$./a.out

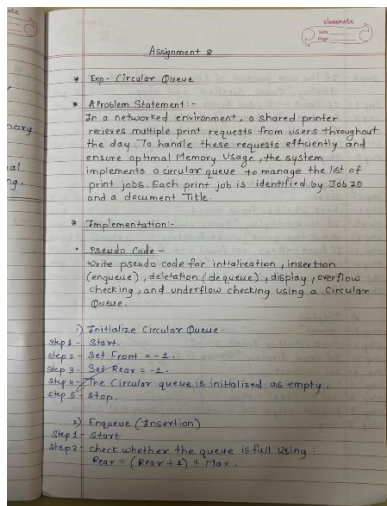
Queue: 10 20 30 40 50

Deleted: 10

Deleted: 20

Queue: 30 40 50

Queue: 30 40 50 60 70



cheatsheet

Step 8 - If the next position of Rear is equal to front, display "Queue Overflow" and stop.

Step 9 - If front $\neq -1$, set front = 0.

Step 10 - Set rear = (Rear + 1) % Max.

Step 11 - Insert the new print job at Rear.

Step 12 - Display "Job inserted successfully".

Step 13 - Stop.

3) Dequeue (Deletion)

Step 1 - Start

Step 2 - Check whether front $\neq -1$

Step 3 - If true, display "Queue Underflow" and stop.

Step 4 - Store the print job at front.

Step 5 - If front $\neq$ Rear, set front = -1 & Rear = -1.

Step 6 - Otherwise, set front = (front + 1) % Max.

Step 7 - Display the deleted/processed print job.

Step 8 - Stop.

4) Display

Step 1 - Start

Step 2 - Check whether front $\neq -1$

Step 3 - If true, display "Queue is Empty" and stop.

Step 4 - Set i = front.

Step 5 - Display the print job at position i.

Step 6 - Set i = (i + 1) % Max.

Step 7 - Repeat steps 5 & 6 until i = (Rear + 1) % Max.

Step 8 - Stop.

cheatsheet

1) Overflow checking -

Step 1 - Start

Step 2 - Check whether (Rear + 1) % Max == front

Step 3 - If true, the circular queue is full.

Step 4 - Display "Queue Overflow! Print queue is full".

Step 5 - Otherwise, insertion is possible.

Step 6 - Stop.

2) Underflow Checking -

Step 1 - Start

Step 2 - Check whether front $\neq -1$

Step 3 - If True, the circular queue is empty.

Step 4 - Display "Queue Underflow! No print jobs available".

Step 5 - Otherwise, deletion is possible.

Step 6 - Stop.

3) Time Complexity -

Operation	Time Comp.
Enqueue	$O(1)$
Dequeue	$O(1)$
Display	$O(n)$
Overflow checking	$O(1)$
Underflow Checking	$O(1)$

cheatsheet

* FAQs

1) What is Circular Queue?

→ A Circular Queue is a queue in which the last position is connected to the first position. It uses memory in a circular manner.

2) Why Circular Queue is preferred over linear queue?

→ To reuse the vacant positions created after deletion and reduces memory wastage.

3) What does a Circular Queue follow?

→ It follows the FIFO (First In, First Out) principle.

4) What are the basic operations performed on a Circular Queue?

→ The basic operations are Enqueue, Dequeue and Display.

5) What is an overflow condition in a Circular Queue?

→ Overflow occurs when the queue is full and we try to insert a new element.

6) What is an underflow condition in a Circular Queue?

→ Underflow occurs when the queue is empty and we try to delete an element.

